Listing 12.4 Registering a tool in an MCP server

```
server.tool(
  "get-chuck-joke",
  "Fetch a random Chuck Norris joke",
  {},
  async () => {
    try {
      const response = await fetch("https://api.chucknorris.io/jokes/random");
      if (!response.ok) {
        return {
          content: [{ type: "text", text: `No joke available
at the moment.` }],
        };
      }
      const data = await response.json();
      return { content:
[{ type: "text", text: data.value }] };
    } catch (err) {
      const message =
err instanceof Error ? err.message : "An unknown error occurred";
      return { content:
[{ type: "text", text: `Error fetching joke: ${message}` }] };
    }
  }
);
```

Registers the tool with its name, description, and input schema

Returns the fetched Chuck Norris fact

This tool fetches a random fun or historical fact about Chuck Norris from the API. The AI can call this tool whenever a user mentions *Chuck*.

DEFINING A PROMPT

Prompts define reusable instructions for the AI model. They can standardize responses, summaries, or transformations on data returned by tools. We use the `server.prompt` method to provide the instructions for the AI model.

Listing 12.5 Creating an MCP prompt to summarize a joke

```
server.prompt(
  "summarize-fact",
  "Summarize a Chuck Norris joke into a short sentence",
  {
    joke: z.string().describe("The joke to summarize"),
  },
  async (args) => {
    const joke = args.joke|| "";

    return {
      messages: [
        {
          role: 'user',
          content: {
            type: 'text',
```

Registers a prompt with input schema

Returns the user message for the AI, instructing it to summarize the provided joke